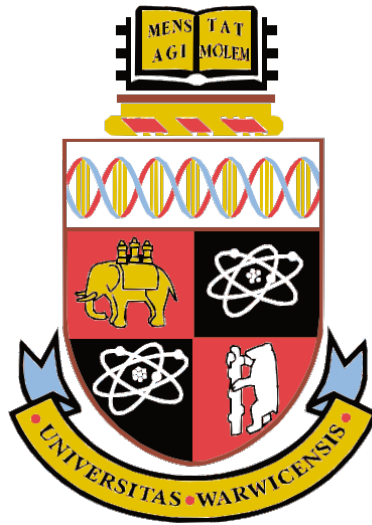




Spectral Methods in Partial Differential Equations

by

Yiyang Hu

MA395 Essay

supervised by

Professor Dwight Barkley

Submitted to The University of Warwick

Mathematics Institute

April, 2026



Contents

1	Introduction	2
2	Pseudospectral method	3
3	Differentiation Matrices: The Fourier Case	6
4	Fast Fourier Transform	9
5	Differentiation Matrices: The Chebyshev Case	13
6	Boundary Value Problems	17
7	Polar Coordinates	25
8	Conclusion	28
A	Fast Fourier Transform revisited	30

1 Introduction

Most partial differential equations are difficult to solve analytically, in the sense that explicit solutions are rarely available. Consequently, it is necessary to develop numerical methods that produce accurate approximations to the true solution. This leads naturally to two fundamental questions of interest in numerical analysis. The first concerns methodology: how can one construct an effective numerical scheme for a given partial differential equation? The second concerns accuracy: in the absence of an exact solution, how can we assess the quality and reliability of the numerical approximation?

In MA3H0 Numerical Analysis & PDE's, we have covered the finite difference method and the finite element method briefly, for partial differential equations. We will recall some basic features about them here, before introducing our new approach: the spectral method.

Consider a one-dimensional spatial interval $I = [a, b]$. In the finite element method, we first partition the interval into a number of N sub-intervals, and choose the $\{\phi_j(x)\}_{j=0}^N$ to be local basis functions which are typically polynomials of fixed degree that are non-zero only over a couple of sub-intervals. Then for an elliptic equation $-(a(x)u')' + b(x)u' + c(x)u = f$, we take the variational formulation to get the weak form of the problem, $B(u, \phi) = l(\phi) \quad \forall \phi \in H_0^1(I)$, where B is a bilinear form associated to the differential operator on $H_0^1(I)$ and l is a linear functional on $H_0^1(I)$. The Finite Element method approximates the solution in a finite-dimensional subspace V_h of H_0^1 , usually spanned by local basis functions, and determines coefficients via the variational formulation.

Suppose that u is the exact solution for the equation and that u_h is the numerical solution. Let $\phi_h \in V_h$. From the above discussion, we know that $B(u, \phi_h) = l(\phi_h)$ and $B(u_h, \phi_h) = l(\phi_h)$. Subtracting these two equations we see that $B(u - u_h, \phi_h) = 0 \quad \forall \phi_h \in V_h$. In other words, we ‘project’ our exact solution u to a finite-dimensional subspace V_h such that the function $u - u_h$ is orthogonal to the finite-dimensional space V_h with respect to the bilinear form B . This is the defining idea of the *Galerkin* method. We will not focus on this method here, though we will see it in later sections for comparison and a detailed discussion can be found in [2, Chapter 3].

The spectral method is similar in philosophy to the finite element method. The major difference is that unlike finite element basis functions, spectral basis functions are typically global, meaning that each basis function influences the approximation across the whole domain. Then our unknown function $u(x)$ can be approximated by a sum of N ‘basis functions’:

$$u(x) \approx u_N(x) = \sum_{n=0}^{N-1} a_n \phi_n(x). \quad (1)$$

When the sum $u_N(x)$ is substituted into the equation $Lu - f(x) = 0$, where L is the differ-

ential operator, like negative Laplacian $-\Delta$, the result is the so-called ‘residual function’ defined by

$$R(x; a_0, a_1, \dots, a_{N-1}) = Lu_N - f.$$

Clearly, we want our ‘residual function’ to be as small as possible in some sense. One possible strategy is not to force the strong residual to vanish pointwise, but instead to impose an orthogonality condition through a variational formulation. This is the idea behind the Galerkin method, which we talked about before.

An alternative strategy is collocation, which we introduce next.

All the code used in this essay to produce the mathematical results can be found in my GitHub repository (<https://github.com/Nickkklos/Spectral-Methods-in-PDEs>). Some of the code is important for demonstrating my understanding of the content, especially regard to the construction of differential matrices. Due to space limitations, I did not explicitly write down every differential matrix. To make it clear, I did not use generative AI for producing the code, but I did use it to help me plot some of the graphs for optimising graphical visualizations.

2 Pseudospectral method

As we mentioned in the Introduction, the goal of the spectral method is to control the residual function $R(x; a_0, \dots, a_{N-1})$ across the spatial domain while satisfying the given boundary conditions. The Pseudospectral method (or collocation) is the strategy we will focus on in this essay, which requires the residual function to be exactly zero at a set of points equal in number to the undetermined coefficients $\{a_n\}_{n=0}^{N-1}$ [2, p. 2]. Therefore, we obtain a system of equations

$$Lu_N(x_i) = f(x_i) \quad \forall i \in \{0, 1, \dots, N-1\}.$$

Together with Equation (1), this yields the collocation matrix system

$$\begin{pmatrix} \phi_0''(x_0) & \phi_1''(x_0) & \cdots & \phi_{N-1}''(x_0) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0''(x_{N-1}) & \phi_1''(x_{N-1}) & \cdots & \phi_{N-1}''(x_{N-1}) \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_{N-1}) \end{pmatrix} \quad (2)$$

where we assume that $L = \frac{d^2}{dx^2}$ is the one-dimensional Laplacian.

However, what is the cost of using ‘global basis’ functions?

We can easily see from Equation (2) that

$$u_N''(x_i) = \sum_{j=0}^{N-1} a_j \phi_j''(x_i) \quad \forall i \in \{0, \dots, N-1\},$$

i.e. the derivative at a specific point x_i is computed using the information of all N points while for the finite difference method, the second derivative is usually computed in the following way:

$$u''(x_i) = \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1}))}{h^2} \quad \forall i \in \{1, \dots, N-2\},$$

where h is the same as defined above and x_i are equally spaced points. Since we only use the nearby points, we will then obtain a tri-diagonal differential matrix that is sparse and computationally cheap. In contrast, the differential matrix we get in Equation (2) is usually dense.

So why do we still bother introducing the Pseudospectral method, given our differential matrix is typically dense hence computationally expensive. To answer this question, let us consider the following example from [5, p. 2], namely the periodic heat equation

$$u_t = u_{xx}, \quad x \in [0, 2\pi], \quad t \geq 0$$

with a periodic boundary condition $u(x, t) = u(x + 2\pi, t)$ and an initial condition $u(x, 0) = u_0(x)$, where $u_0(x)$ is periodic. Since the exact solution u is periodic, it can be written as an infinite Fourier series, then we define the Fourier approximate solution $u_N(x)$ as

$$u_N(x) = \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} a_k(t) e^{ikx}, \quad x \in [0, 2\pi], \quad (3)$$

where each $a_k(t)$ is to be determined.

Plug into the heat equation, we obtain the following:

$$\sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} a_k'(t) e^{ikx} = \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} a_k(t) (ik)^2 e^{ikx}.$$

Since frequencies are uncoupled, we have $a_k'(t) = -k^2 a_k(t)$, which gives

$$a_k(t) = e^{-k^2 t} a_k(0),$$

where the values $a_k(0)$ are determined by using the initial function:

$$a_k(0) = \frac{1}{2\pi} \int_0^{2\pi} u_0(x) e^{-ikx} dx.$$

It is clear that the error of approximation is

$$\begin{aligned} |u(x, t) - u_N(x, t)| &\leq \left| \sum_{|k| \geq N/2} a_k(0) e^{ikx} e^{-k^2 t} \right| \\ &\leq \sup_k |a_k(0)| \sum_{|k| \geq N/2}^{\infty} e^{-k^2 t} \\ &\leq 2 \sup_{0 \leq x \leq 2\pi} |u_0(x)| \int_{N/2}^{\infty} e^{-x^2 t} dx. \end{aligned}$$

N	J(0.1, N/2)	J(0.5, N/2)	J(1, N/2)	N	Error
4	1.04	0.057026	0.0041455	10	0.014148
6	0.50364	0.0033837	1.9577e-05	20	0.0041025
8	0.20637	7.9388e-05	1.3663e-08	30	0.0018833
10	0.071036	7.1853e-07	1.3625e-12	40	0.0010504
12	0.020431	2.473e-09	1.9071e-17	50	0.00067286
14	0.0048907	3.208e-12	3.7078e-23	60	0.00047127

Figure 1: Left: Comparison of the results of $J(t, N)$ for different N and t . Right: Error computed by FD

Therefore the error goes to zero very quickly as N becomes reasonably large. The convergence rate is determined by the integral term

$$J(t, N) := \int_N^{\infty} e^{-x^2 t} dx = \sqrt{\frac{\pi}{4t}} \operatorname{erfc}(\sqrt{t}N),$$

where $\operatorname{erfc}(x)$ is the complementary error function.

From Figure 1, it is clear that we can basically achieve 10^{-12} with only 10 points $(x_0, \dots, x_9)$ at $t = 1$ for this smooth periodic test problem. In contrast, we see that the error is still around $O(10^{-4})$ even with 60 points in the Finite Difference method, subject to $u_0(x) = \sin(x)$.

Therefore, although our differential matrices for the Pseudospectral method are usually not sparse, the required dimension to reach such a high accuracy turns out to be extremely small. This kind of behavior is called *spectral accuracy*, which we will see again and again in later sections.

3 Differentiation Matrices: The Fourier Case

Inspired by our last example, we now consider how to compute a generic differential matrix for periodic problems, that is problems posed on a periodic grid.

In this section, our spatial domain will be $I = [0, 2\pi]$. Recall Equation (1), since our function $u(x)$ is periodic in this setting, our ‘global basis’ functions $\phi_n(x)$ are obviously taken to be trigonometric functions or exponential functions e^{inx} . It is not too hard, especially in the polynomial case which we will encounter later, to see that we can rewrite Equation (1) as

$$u_N(x) = \sum_{n=1}^N u(x_n) F_n(x), \quad (4)$$

in the collocation setting, where $F_n(x)$ are called *cardinal* functions which satisfy

$$F_n(x_j) = \begin{cases} 0 & j \neq n, \\ 1 & j = n. \end{cases}$$

Then equation (2) will change into

$$\begin{pmatrix} F_1''(x_1) & F_2''(x_1) & \cdots & F_N''(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ F_1''(x_N) & F_2''(x_N) & \cdots & F_N''(x_N) \end{pmatrix} \begin{pmatrix} u(x_1) \\ u(x_2) \\ \vdots \\ u(x_N) \end{pmatrix} = \begin{pmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_N) \end{pmatrix} \quad (5)$$

So, our next goal is to find such cardinal functions.

The cardinal functions depend on the parity of N , and we will give the formulae directly here, and only prove the case when N is even later.

$$F_n(x) = \frac{1}{N} \sin\left(\frac{N}{2}(x - x_n)\right) \cot\left(\frac{x - x_n}{2}\right), \quad N \text{ is even}, \quad (6)$$

$$F_n(x) = \frac{1}{N} \sin\left(\frac{N}{2}(x - x_n)\right) \csc\left(\frac{x - x_n}{2}\right), \quad N \text{ is odd}. \quad (7)$$

where $x_n = n\frac{2\pi}{N}$. The *differential matrices* $D^{(m)}$ with entries $D_{jn}^{(m)} = F_n^{(m)}(x_j)$ for N

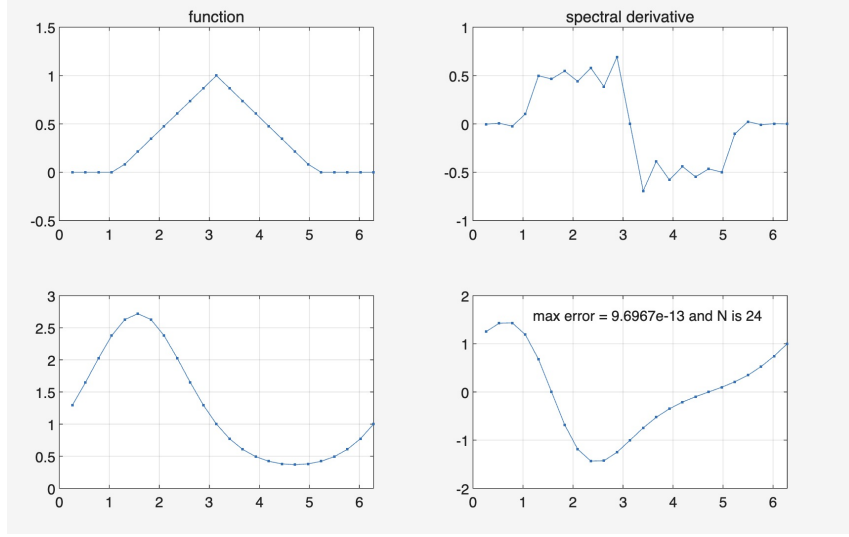


Figure 2: Spectral differentiation of a rough function and a smooth function. The smooth function gives 12-digit accuracy using only 24 points

being even and odd are defined as follows [8]:

N is even,

$$D_{jn}^{(1)} = \begin{cases} 0, & j = n, \\ \frac{1}{2}(-1)^{j-n} \cot\left(\frac{(j-n)h}{2}\right), & j \neq n, \end{cases} \quad D_{jn}^{(2)} = \begin{cases} -\frac{\pi^2}{3h^2} - \frac{1}{6}, & j = n, \\ -\frac{1}{2}(-1)^{j-n} \csc^2\left(\frac{(j-n)h}{2}\right), & j \neq n, \end{cases}$$

N is odd,

$$D_{jn}^{(1)} = \begin{cases} 0, & j = n, \\ \frac{1}{2}(-1)^{j-n} \csc\left(\frac{(j-n)h}{2}\right), & j \neq n, \end{cases} \quad D_{jn}^{(2)} = \begin{cases} -\frac{\pi^2}{3h^2} + \frac{1}{12}, & j = n, \\ -\frac{1}{2}(-1)^{j-n} \csc\left(\frac{(j-n)h}{2}\right) \cot\left(\frac{(j-n)h}{2}\right), & j \neq n. \end{cases}$$

where $h = \frac{2\pi}{N}$. To illustrate the spectral accuracy of Fourier differentiation discussed earlier, we consider the following two numerical examples adapted from [5].

In Figure 2, the upper-left function is the hat function $\max(0, 1 - |x - \pi|/2)$, which is not differentiable at three isolated points, while the lower-left function is the smooth function $e^{\sin x}$. We can easily see from the two graphs on the right that the Pseudospectral method works extremely well with the smooth function $e^{\sin x}$ but is much less effective for the nonsmooth one. Together with Figure 3, this confirms that although the differential matrix for Pseudospectral methods is usually dense, we can achieve much higher accuracy compared to the finite difference method using the same small number of points.

To close this subsection, we will use the differential matrix we just derived to solve a partial differential equation. Consider the following variable coefficient transport equation [6,

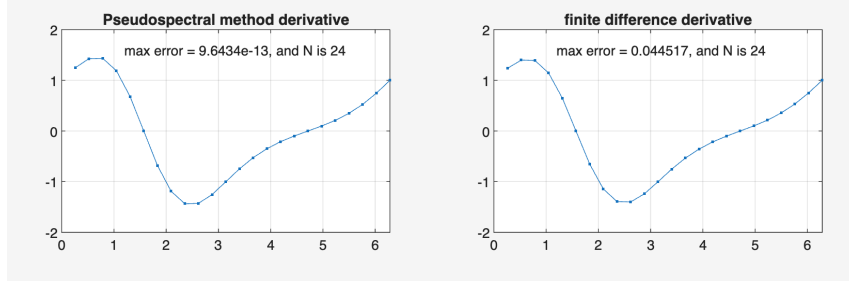


Figure 3: Comparison between Pseudospectral method and finite difference

p. 24]

$$u_t + c(x)u_x = 0, \quad c(x) = \frac{1}{5} + \sin^2(x-1) \quad (8)$$

for $x \in [0, 2\pi], t > 0$ with periodic boundary condition. Regarding the initial condition $u_0(x)$, we take $u_0(x) = e^{-100(x-1)^2}$. Although this function is not mathematically periodic, it can be treated numerically as periodic because it is smooth and exponentially small near the endpoints.

To construct our numerical scheme, we proceed just as we might with a finite difference approach of a PDE. For the time derivative we use Backward-Euler to guarantee the stability, and we approximate the spatial derivative spectrally. Let $v^{(n)} = (v_1^{(n)}, v_2^{(n)}, \dots, v_N^{(n)})$ be the vector at time step n that approximates $u(x, n\Delta t)$ at grid points x_j , and our spectral derivative at these points is $(D^{(1)}v^{(n)})_j$, where $D^{(1)}$ is the same as we defined before. The approximation becomes

$$\frac{v_j^{(n+1)} - v_j^{(n)}}{\Delta t} = -c(x_j)(D^{(1)}v^{(n+1)})_j, \quad j \in \{1, 2, \dots, N\}. \quad (9)$$

The resulting linear system is $(I + \Delta t C D^{(1)})v^{(n+1)} = v^{(n)}$ with $C = \text{diag}(c(x_j))$. We can construct a similar numerical scheme using the finite difference method as follows:

$$\frac{v_j^{(n+1)} - v_j^{(n)}}{\Delta t} = -c(x_j)(D_{FD}v^{(n+1)})_j, \quad j \in \{1, 2, \dots, N\},$$

where D_{FD} is the circulant tri-diagonal matrix defined as

$$D_{FD} = \frac{1}{2h} \begin{pmatrix} 0 & -1 & 0 & \cdots & 1 \\ -1 & 0 & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & -1 & 0 & 1 \\ 1 & \cdots & 0 & -1 & 0 \end{pmatrix}$$

The spurious dispersion observed in Figure 5 is a manifestation of phase error. In a centred finite-difference discretisation, the exact wavenumber k is replaced by a numerical

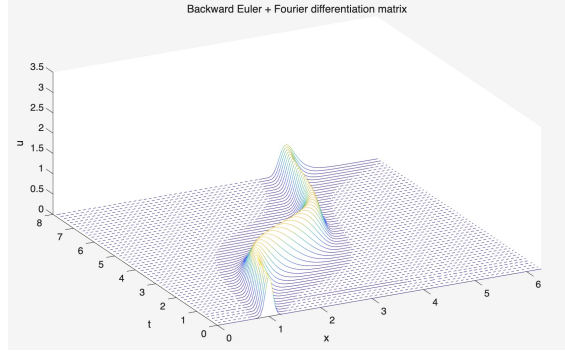


Figure 4: Transport equation (8) solved by the Pseudospectral method and Backward-Euler

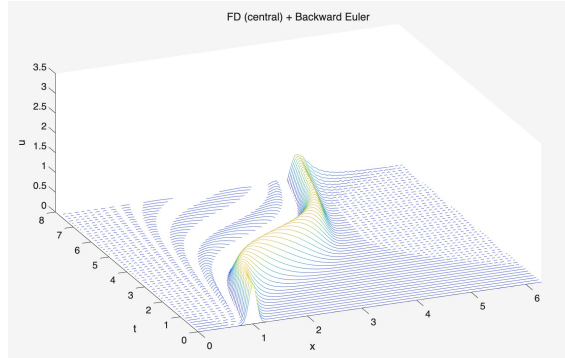


Figure 5: Transport equation (8) solved by the finite difference method and Backward-Euler

wavenumber $K = \sin(kh)/h$, which leads to incorrect phase velocities for different Fourier modes. As a result, non-physical dispersive artefacts accumulate over time. In contrast, the pseudospectral method differentiates Fourier modes exactly and is therefore essentially free of phase error, which explains the absence of spurious dispersion in the spectral solution (Figure 4) and constitutes one of the most conspicuous advantages of spectral methods [6, p. 27].

This observation motivates the next section on the Fast Fourier Transform, which enables the efficient implementation of spectral methods and elevates them to a level of practical importance comparable to that of the finite-difference and the finite-element methods in modern numerical analysis.

4 Fast Fourier Transform

In scientific computing, the differential matrix we derived in the previous section has complexity $O(N^2)$. Although in our examples the N is usually small, in practice, it still needs a large number of points to achieve a high accuracy (arguably, $O(10^{-12})$). In contrast, the sparse matrix like the tri-diagonal matrix D_{FD} we defined earlier has only

complexity $O(N)$. Thus the spectral method had not been used widely for a long time until the discovery of the Fast Fourier Transform (FFT) by Cooley and Tukey [3]. We will absorb the notation used in [6, p. 3] and our N will always be even and $h = \frac{2\pi}{N}$ in this section.

Let us recall the idea of ‘global basis’ functions and the heat equation we discussed earlier. We said since our function is periodic with 2π -periodicity, we can write it as an infinite Fourier series.

$$u(x) = \sum_{k=-\infty}^{\infty} a_k e^{ikx} \quad (10)$$

where a_k is defined as

$$a_k = \frac{1}{2\pi} \int_0^{2\pi} u(x) e^{-ikx} dx$$

If we now pass from the continuous Fourier expansion to a discrete representation based on N grid values, it is not immediately obvious why the mode set in Equation (11) is the natural discrete analogue of the truncated Fourier expansion Equation (3).

$$u_N(x) = \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}} \hat{u}_k e^{ikx} \quad (11)$$

In our setting, we are dealing with a bounded spatial domain $I = [0, 2\pi]$, and the function on this domain is assumed to be periodic. Given only N points $\{x_1, x_2, \dots, x_N\}$ and information on these points, we need to slightly change our view of functions being equal.

Two complex exponentials $f(x) = e^{ik_1x}$ and $g(x) = e^{ik_2x}$ are equal on $\mathbb{R}$ iff $k_1 = k_2$, but here we only need functions to be equal at these N points as we only have information on these points. In other words, $e^{ik_1x_j} = e^{ik_2x_j}, \forall j \in \{1, 2, \dots, N\}$. It follows that for any complex exponential e^{ikx} , there are infinitely many other complex exponential that match it on these grid points — ‘aliases’ of k , i.e. $k_1 - k_2$ is an integer multiple of $\frac{2\pi}{h}$. Thus Fourier modes are identified modulo $N = \frac{2\pi}{h}$ in the discrete setting.

Next question is how to determine $\hat{u}_k$ in this discrete setting?

Since we are not in the general setting of Fourier Analysis, the former formula for a_k obviously does not work. Instead, we will introduce so-called *Discrete Fourier transform* (DFT).

The formula for the DFT is ([6])

$$\hat{u}_k = h \sum_{j=1}^N e^{-ikx_j} u(x_j), \quad k = -\frac{N}{2} + 1, \dots, \frac{N}{2}, \quad (12)$$

and the *inverse* DFT is given by

$$u(x_j) = \frac{1}{2\pi} \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}} e^{ikx_j} \hat{u}_k, \quad j = 1, \dots, N. \quad (13)$$

Similar to the formula of Fourier transform, it does not really matter where we put 2π as long as we can keep consistent.

Right now we have the tool to show the existence of the cardinal functions in the last section, so let us take a detour before going into the climax of this section.

Recall our goal is to find cardinal functions $F_n(x)$ such that $F_n(x_j) = 1$ iff $n = j$ and 0 otherwise.

Definition 1. (*Band-limited function*) [2, p. 343] A function $f_W(y)$ is said to be band-limited with bandwidth W if it can be represented as the truncated Fourier integral

$$f_W(y) = \frac{1}{2\pi} \int_{-W}^W F(w) e^{-iwy} dw.$$

So here, in the discrete setting our band-limited function is defined as

$$p(x) = \frac{1}{2\pi} \sum'_{k=-\frac{N}{2}} e^{ikx} \hat{u}_k, \quad x \in [0, 2\pi], \quad (14)$$

where

$$\begin{aligned} \sum'_{k=-\frac{N}{2}} e^{ikx} \hat{u}_k &= \frac{1}{2} e^{-\frac{N}{2}ix} \hat{u}_{-\frac{N}{2}} + \frac{1}{2} e^{\frac{N}{2}ix} \hat{u}_{\frac{N}{2}} + \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}-1} e^{ikx} \hat{u}_k \\ &= e^{\frac{N}{2}ix} \hat{u}_{\frac{N}{2}} + \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}-1} e^{ikx} \hat{u}_k \quad -N/2 = N/2 \text{ in discrete setting} \\ &= \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}} e^{ikx} \hat{u}_k \end{aligned}$$

The reason for this modification is a bit subtle. The wavenumber at $k = \frac{N}{2}$ is special: on the grid $x_j = \frac{2\pi j}{N}$, $e^{i\frac{N}{2}x_j} = (-1)^j$ is real. If one were to apply the usual Fourier multiplier ik at $k = \frac{N}{2}$, the derivative is $i\frac{N}{2}(-1)^j$ which is purely imaginary. Thus we fix this issue by making our index symmetric, and this wavenumber is known as *Nyquist* frequency.

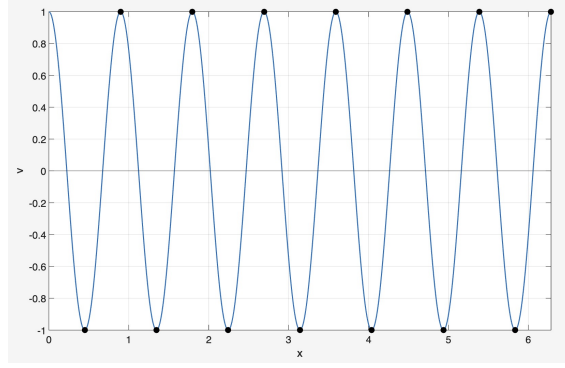


Figure 6: The sawtooth function and the black dots are the equally spaced grid. The derivative at every grid is zero that implies the spectral derivative is also zero.

It is easy to see that $p(x_j) = u(x_j)$ for each j , so in the discrete setting we could claim $p(x) = u_N(x)$. Consider the delta function $\delta_j := \begin{cases} 1, & j \equiv 0 \pmod{N}, \\ 0, & j \not\equiv 0 \pmod{N}. \end{cases}$ If we know the band-limited function $S_N(x)$ of δ_j then we have $u(x_N)S_N(x)$ equals to $u(x)$ at x_N and equals 0 at all other grid. Then we can expand u as a linear combination of translated delta functions.

Plugging into Equation (12) we get $\hat{\delta}_j = h$ for each k , and so from Equation (14), we get

$$\begin{aligned}
 S_N(x) &= \frac{h}{2\pi} \sum'_{k=-\frac{N}{2}}^{\frac{N}{2}} e^{ikx} \hat{u}_k = \frac{h}{2\pi} \left(\frac{1}{2} \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} e^{ikx} + \frac{1}{2} \sum_{k=-\frac{N}{2}+1}^{\frac{N}{2}} e^{ikx} \right) \\
 &= \frac{h}{2\pi} \cos\left(\frac{x}{2}\right) \sum_{k=-\frac{N}{2}+\frac{1}{2}}^{\frac{N}{2}-\frac{1}{2}} e^{ikx} \\
 &= \frac{h}{2\pi} \cos\left(\frac{x}{2}\right) \frac{e^{i(-\frac{N}{2}+\frac{1}{2})x} - e^{i(\frac{N}{2}+\frac{1}{2})x}}{1 - e^{ix}} \\
 &= \frac{h}{2\pi} \cos\left(\frac{x}{2}\right) \frac{e^{i(-\frac{N}{2})x} - e^{i(\frac{N}{2})x}}{e^{i(-\frac{x}{2})} - e^{i(\frac{x}{2})}} \\
 &= \frac{h}{2\pi} \cos\left(\frac{x}{2}\right) \frac{\sin(\frac{Nx}{2})}{\sin(\frac{x}{2})}
 \end{aligned}$$

where $S_N(x)$ is known as *periodic sinc function*. Thus one can check that Equation (4) is actually the same as

$$p(x) = \sum_{j=1}^N u(x_j) S_N(x - x_j),$$

and $S_N(x)$ is identical with $F_n(x)$ we defined in Equation (6).

With the help of DFT, there is no need to compute the differential matrix explicitly anymore, what we do now is

- Given the grid value of u at grid, compute $\hat{u}$.
- Define $\hat{w}_k = (ik)^\nu \hat{v}_k$, but $\hat{w}_{\frac{N}{2}} = 0$ if ν is odd
- Compute w from $\hat{w}_k$

One might observe that we actually have not simplified our computation by introducing DFT. Although it is obvious that differentiating $u_N(x)$ once is just the same as multiplying ik once in front of Equation (11). It seems easier than before, but the complexity of determining a_k in Equation (11) (or $\hat{u}_k$ in Equation (12)) is still $O(N^2)$ from the construction.

At first sight, the discussion above may seem inefficient. This is not the case, however. Nevertheless, the complexity can be reduced to $O(N \log N)$ if we use Fast Fourier Transform (FFT). In MATLAB, this is implemented using the commands `fft` and `ifft`, which denote the fast Fourier transform and inverse fast Fourier transform, respectively. The basic idea is analogous to recursive bisection, in that the problem is repeatedly divided into smaller subproblems. Since our main focus is on the realisation of the spectral method, we leave the details of this topic in the Appendix.

5 Differentiation Matrices: The Chebyshev Case

Of course, not all problems can be treated as periodic. We now begin to consider how to construct spectral methods for bounded, non-periodic domains.

Suppose that we wish to work on $[-1, 1]$ with non-periodic functions. One approach would be to pretend that the functions were periodic and use Fourier interpolation in equally spaced points. It is a method that works fine for problems whose solutions are exponentially close to a constant near the boundaries like the transport equation (8) we discussed earlier. In general, however, this approach will sacrifice the accuracy advantages of spectral methods. If a smooth non-periodic function on $[-1, 1]$ is extended periodically, the extension will in general develop endpoint discontinuities or derivative jumps. As a result, Fourier interpolation loses spectral accuracy and may exhibit Gibbs oscillations near the endpoints, known as the Gibbs phenomenon. The error in the interpolant will be $O(1)$, the error in the first derivative will be $O(N)$, and so on [6, p. 42].

Therefore, it is customary to replace complex exponentials we used in the previous two sections by algebraic polynomials, $p(x) = a_0 + a_1x + \dots + a_Nx^N$. Then we might follow the same procedure, interpolating the function at grid points that are equally spaced. However, this turns out to be catastrophically unstable, and is known as the *Runge phenomenon* which is more extreme than the Gibbs phenomenon.

Instead, we will use *Gauss-Chebyshev-Lobatto* points (15). A rigorous justification for this optimal distribution via potential theory is beyond the scope of this essay, but detailed proofs can be found in [2], and we just take it for granted here. An example from [6, p. 42]

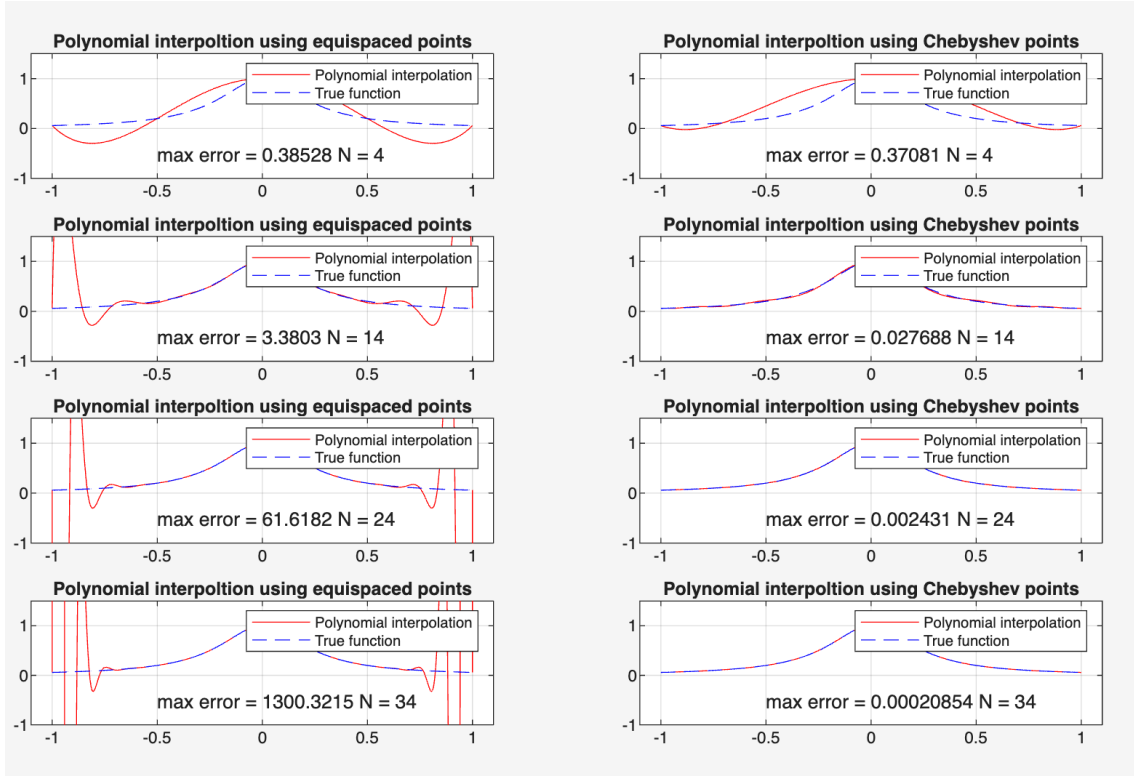


Figure 7: Degree N interpolation of $u(x) = \frac{1}{1+16x^2}$ in $N + 1$ points. With N increasing, the errors increase exponentially in the equally spaced case —*the Runge phenomenon*— whereas in the Chebyshev case they decrease exponentially.

shows that Chebyshev points indeed work well on polynomial interpolation (see Figure 7).

$$x_j = \cos\left(\frac{j\pi}{N}\right), \quad j = 0, 1, \dots, N \quad (15)$$

Geometrically, we can visualize these points as the projections on $[-1, 1]$ of equally spaced points on the upper half of the unit disk, as sketched in Figure (8). It is easily seen that the points are clustered at the boundaries, which helps achieve stable polynomial interpolation and enables spectral accuracy for sufficiently regular functions, and the reasons for these are about *potential theory* (see [6, chapter. 5]).

So how to compute the differential matrices in this setting?

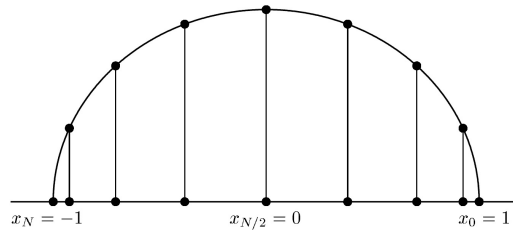


Figure 8: Chebyshev points [6]

Proceeding as in the Fourier case, we actually need to find the cardinal functions $f_k(x)$ that are polynomials of degree at most N such that

$$u \approx u_N(x) = \sum_{k=0}^N u(x_k) f_k(x). \quad (16)$$

The cardinal functions are quite simple to compute this time, which we have already covered in MA266 Multilinear Algebra and MA2K4 Numerical Methods and Computing. They are known as *Lagrange polynomials* defined as

$$f_k = \frac{1}{a_k} \prod_{j=0, j \neq k}^N (x - x_j), \quad a_k = \prod_{j=0, j \neq k}^N (x_k - x_j) \quad (17)$$

Take the logarithm and differentiate to obtain

$$f'_j(x) = f_j(x) \sum_{k=0, k \neq j}^N (x - x_k)^{-1}$$

By analogy with the collocation formulation above, we define the first-order differentiation matrix by

$$D_{ij} = f'_j(x_i) = \frac{a_i}{a_j(x_i - x_j)}, \quad i \neq j$$

and

$$D_{jj} = f'_j(x_j) = f_j(x_j) \sum_{k=0, k \neq j}^N (x_j - x_k)^{-1} = \sum_{k=0, k \neq j}^N (x_j - x_k)^{-1}$$

Plug $x_j = \cos(\frac{j\pi}{N})$ into the above formulae. The differential matrix for Chebyshev points is at our disposal:

$$D_{ij} = \frac{c_i(-1)^{(i+j)}}{c_j(x_i - x_j)} \quad i \neq j, \quad D_{ii} = -\frac{x_i}{2(1 - x_i^2)} \quad 1 \leq i \leq N-1$$

$$D_{00} = \frac{2N^2 + 1}{6}, \quad D_{NN} = -\frac{2N^2 + 1}{6}$$

with $c_j = 2$ if $j = 0, N$ or $c_j = 1$ for other j . Similarly, we can find D_{ii} by plugging.

In addition, for Chebyshev collocation one has

$$D^{(m)} = (D^{(1)})^m.$$

This contrasts with the Fourier case on an even periodic grid, where the Nyquist mode must be treated separately. Indeed, let

$$x_j = \frac{2\pi j}{N}, \quad j = 1, \dots, N,$$

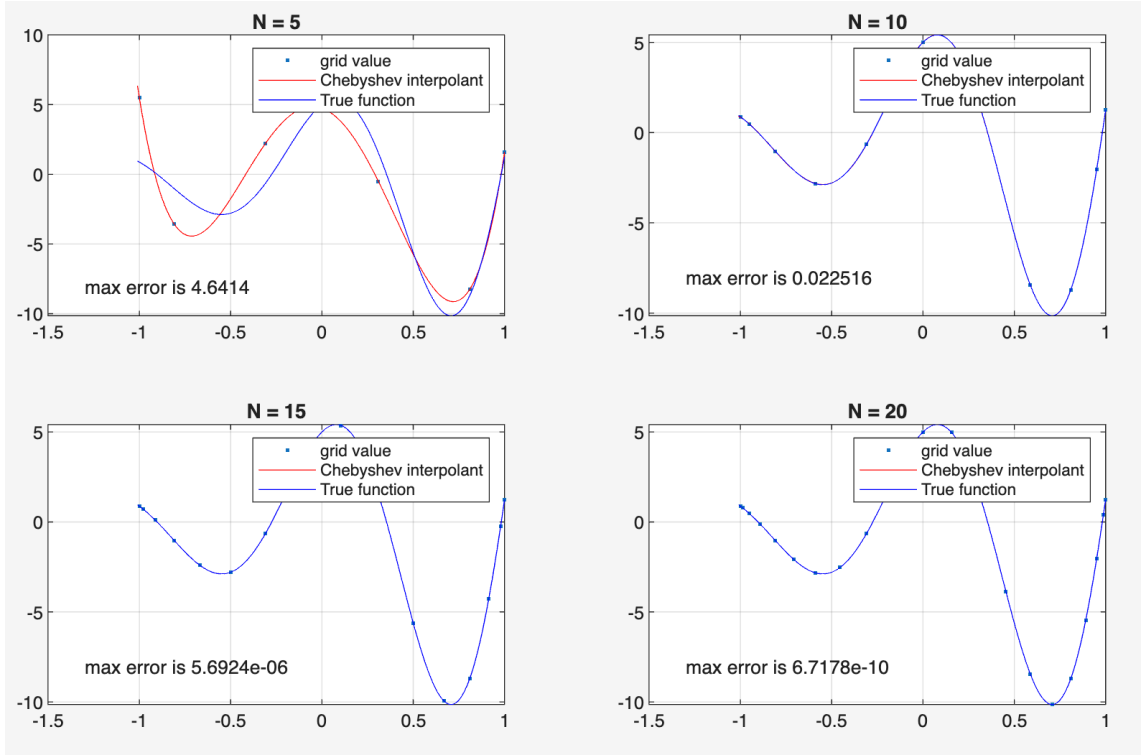


Figure 9: Chebyshev differentiation of $u(x) = e^x \sin(5x)$

and consider the grid function

$$v_j = \cos\left(\frac{Nx_j}{2}\right) = (-1)^j.$$

Then, by the convention used for the Nyquist mode in the first-derivative matrix, we have

$$D^{(1)}v = 0,$$

whereas

$$D^{(2)}v = -\left(\frac{N}{2}\right)^2 v.$$

Hence, in the Fourier case with even N ,

$$D^{(2)} \neq (D^{(1)})^2$$

in general.

Let us first see how accurate the spectral method is for a non-periodic function $e^x \sin(5x)$ (see Figure 9). The outputs show with only $N = 20$ we can get nine-digit accuracy.

However, similar to the periodic case, this does not work equally well with general func-

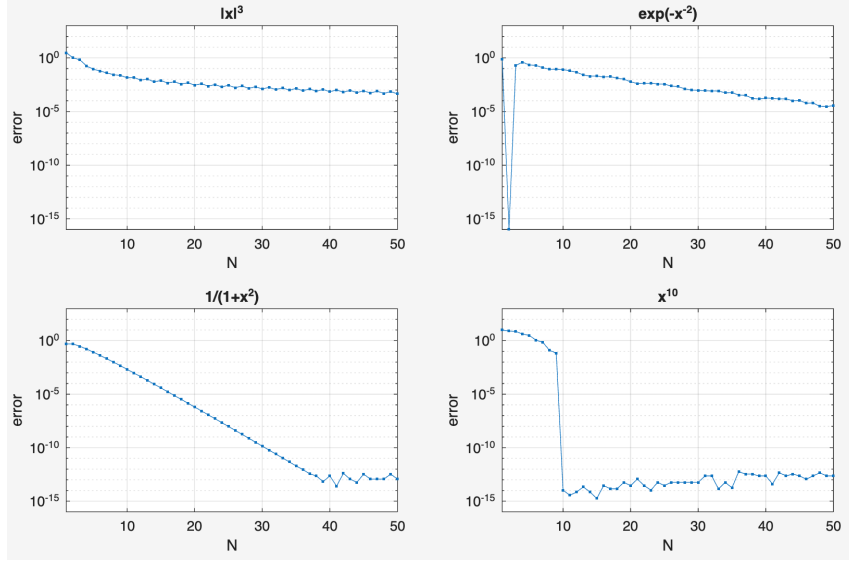


Figure 10: Accuracy of the Chebyshev spectral derivative for four functions of increasing smoothness

tions. Consider the following four examples:

$$u_1 = |x|^3, \quad u_2(x) = \begin{cases} e^{-1/x^2}, & x \neq 0, \\ 0, & x = 0, \end{cases} \quad u_3 = \frac{1}{1+x^2}, \quad u_4 = x^{10}$$

The first function u_1 has a third derivative of bounded variation (hence only finite smoothness), the second one u_2 is C^∞ but is not analytic at the origin, the third one u_3 is analytic in a neighborhood of $[-1, 1]$, and the last one u_4 is a polynomial of degree 10 (hence entire). In Figure 10 we can easily see the convergence rate improves from algebraic to super-algebraic and then to exponential as the function regularity increases from finite smoothness to C^∞ and analyticity.

6 Boundary Value Problems

We have defined the Chebyshev differentiation matrix D_N and shown that it works as expected. In this section, we illustrate how such matrices can be used to solve some boundary value problems arising in ODEs and PDEs. Most examples in this section are adapted from [6]. Some of the codes have been slightly modified for our purposes, and consequently the resulting figures are not exactly the same as those in the original source.

Homogeneous Dirichlet Boundary Condition

Having discussed only the one-dimensional case so far, we begin with the linear ODE

$$\begin{array}{c} \text{ignored} \rightarrow \end{array} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ \vdots \\ w_{N-1} \\ w_N \end{pmatrix} = \begin{pmatrix} \text{gray} & \text{white} & \text{gray} \\ & D_N^2 & \\ \text{gray} & & \text{gray} \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \\ \vdots \\ \vdots \\ v_{N-1} \\ v_N \end{pmatrix} \begin{array}{c} \leftarrow \text{zeroed} \\ \\ \\ \\ \leftarrow \text{zeroed} \end{array}$$

Figure 11: Differential matrix with homogeneous Dirichlet Boundary condition. The interior (white part) of the matrix D_N^2 is $\tilde{D}_N^2$ with size $(N-1) \times (N-1)$ [6, p. 62]

boundary value problem

$$u_{xx} = e^{4x}, \quad -1 < x < 1, \quad u(\pm 1) = 0. \quad (18)$$

This is a one-dimensional Poisson equation, with solution $u(x) = \frac{e^{4x} - x \sinh(4) - \cosh(4)}{16}$.

To solve this problem numerically, we first form the second-order differential matrix $D^{(2)}$ of u . As we discussed earlier, it is the square of $D^{(1)}$. Second, u needs to satisfy the boundary conditions. For simple problems like (18) with homogeneous Dirichlet boundary conditions, we can proceed as follows. We take interior Chebyshev points $x_1, x_2, \dots, x_{N-1}$ as our computational grid, with $v = (u(x_1), \dots, u(x_{N-1}))^T$. Spectral differentiation is then carried out like this:

- Let $p(x)$ be the polynomial of degree at most N with $p(\pm 1) = 0$ and $p(x_j) = u(x_j)$, $1 \leq j \leq N-1$.
- $w_j = p''(x_j)$, $1 \leq j \leq N-1$.

This is not the only means of imposing boundary conditions in spectral methods. We will consider alternatives later in this section.

Now $D_N^{(2)}$ is an $(N+1) \times (N+1)$ matrix that maps $(u(x_0), \dots, u(x_N))$ to a vector $(u''(x_0), \dots, u''(x_N))$. The procedure just described amounts to a decision that we wish to:

- Fix $u(x_0) = u(x_N) = 0$.
- Ignore w_0 and w_N

This implies that the first and last columns of $D^{(2)}$ have no effect (since multiplied by zero), and the first and last rows have no effect either (since w_0 and w_N are ignored). In other words, to solve this one-dimensional Poisson problem by a Chebyshev differential matrix, we just need to use the interior block of $D^{(2)}$, known as $\tilde{D}^{(2)}$. (see Figure 11).

As illustrated in Figure 12, the spectral method achieves approximately ten-digit accuracy using only 17 grid points ($N = 16$). In contrast, the finite difference method attains only about four-digit accuracy even with 129 grid points ($N = 128$), as shown in Figure 13.

How can we extend this method to boundary value problems in several space dimensions? To be specific, here is a two-dimensional Poisson problem with exact solution $u(x, y) =$

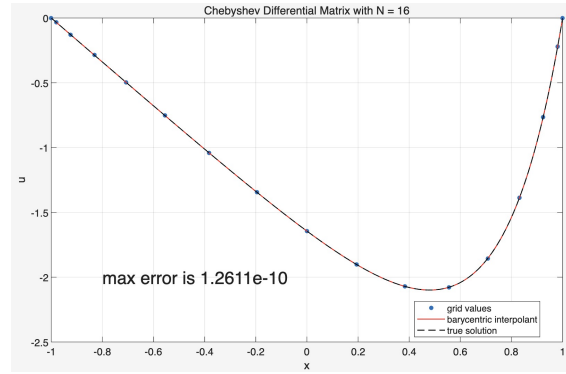


Figure 12: Solution of the linear boundary value problem (18) using the Pseudospectral method

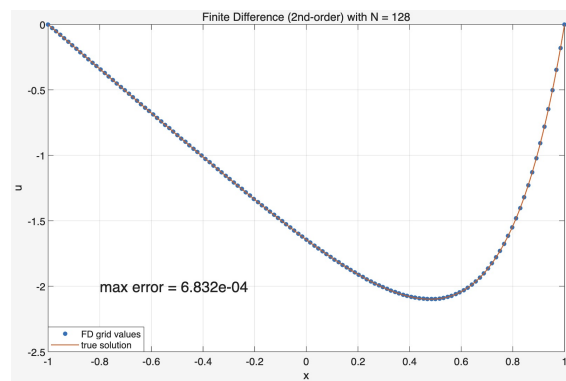


Figure 13: Solution of the linear boundary value problem (18) using the Finite difference

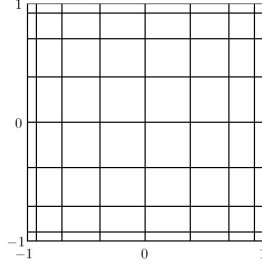


Figure 14: A tensor product grid [6, p. 67]

$\sin(\pi x) \sin(\pi y)$:

$$u_{xx} + u_{yy} = -2\pi^2 \sin(\pi x) \sin(\pi y), \quad -1 < x, y < 1, \quad u = 0 \text{ on } \partial\Omega \quad (19)$$

For such a problem we naturally set up a grid based on Chebyshev points independently in each direction, called a *tensor product grid* (Figure 14). As in the one-dimensional case, we can see that it is also clustered near the boundary, which is useful for dealing with boundary layers or other features that appear around the boundary.

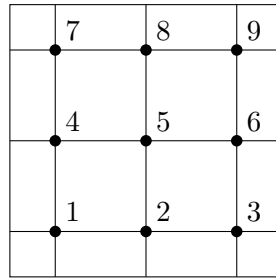
To solve the one-dimensional problem, we employ a differentiation matrix. For the two-dimensional problem, the standard approach is to construct the discretised operator via tensor (Kronecker) products.

The Kronecker product of two matrices A and B is denoted by $A \otimes B$ and is computed in MATLAB by the command `kron(A,B)` [6, p. 68]. For example,

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} a & b & 2a & 2b \\ c & d & 2c & 2d \\ 3a & 3b & 4a & 4b \\ 3c & 3d & 4c & 4d \end{pmatrix}$$

Now to explain how Kronecker products can be applied to spectral methods, let us consider the case $N = 4$.

Suppose we number the internal nodes in this ordering:



Also suppose we have data $v = (v_1, \dots, v_9) = (u(x_1), \dots, u(x_9))$ at these grid points. In

other words, we are storing the grid values row by row. (Here we use row-wise ordering of the unknowns; with MATLAB's default column-wise vectorisation, the Kronecker factors would be reversed. Details can be found in my corresponding GitHub code) We aim to find the Laplacian by differentiating spectrally in x and y directions independently. In other words,

$$v_j^{xx} = \sum_{k=1}^9 a_k v_k, \quad v_j^{yy} = \sum_{k=1}^9 b_k v_k, \quad j \in 1, \dots, 9.$$

Thus the second derivative with respect to x at 1, 2, 3 should be the same as the second Chebyshev differential matrix $\tilde{D}_4^{(2)}$ defined before, i.e.

$$\begin{pmatrix} v_1^{xx} \\ v_2^{xx} \\ v_3^{xx} \end{pmatrix} = \tilde{D}_4^2 \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}.$$

The same holds for v_4, v_5, v_6 , and v_7, v_8, v_9 , and we obtain the following differential matrix on the x direction:

$$\begin{pmatrix} \tilde{D}_4^{(2)} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \tilde{D}_4^{(2)} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \tilde{D}_4^{(2)} \end{pmatrix} \quad (20)$$

Similarly, in the y direction, the second derivative at 1, 4, 7 should be the same as the second Chebyshev differential matrix $\tilde{D}_4^{(2)}$, i.e.

$$\begin{pmatrix} v_1^{yy} \\ v_4^{yy} \\ v_7^{yy} \end{pmatrix} = \tilde{D}_N^2 \begin{pmatrix} v_1 \\ v_4 \\ v_7 \end{pmatrix}$$

then the corresponding matrix is

$$\begin{pmatrix} -14 & 0 & 0 & 6 & 0 & 0 & -2 & 0 & 0 \\ 0 & -14 & 0 & 0 & 6 & 0 & 0 & -2 & 0 \\ 0 & 0 & -14 & 0 & 0 & 6 & 0 & 0 & -2 \\ 4 & 0 & 0 & -6 & 0 & 0 & 4 & 0 & 0 \\ 0 & 4 & 0 & 0 & -6 & 0 & 0 & 4 & 0 \\ 0 & 0 & 4 & 0 & 0 & -6 & 0 & 0 & 4 \\ -2 & 0 & 0 & 6 & 0 & 0 & -14 & 0 & 0 \\ 0 & -2 & 0 & 0 & 6 & 0 & 0 & -14 & 0 \\ 0 & 0 & -2 & 0 & 0 & 6 & 0 & 0 & -14 \end{pmatrix} \quad (21)$$

Notice that (20) is equal to $I \otimes \tilde{D}_4^{(2)}$, and that (21) is equal to $\tilde{D}_4^{(2)} \otimes I$.

Figure 15 shows exponential convergence of the Chebyshev spectral method for the 2D

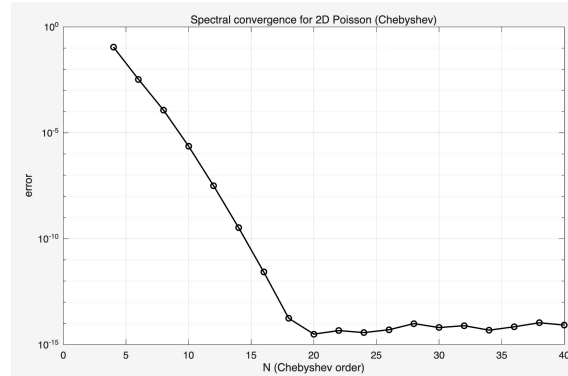


Figure 15: Convergence rate of the Poisson equation (19)

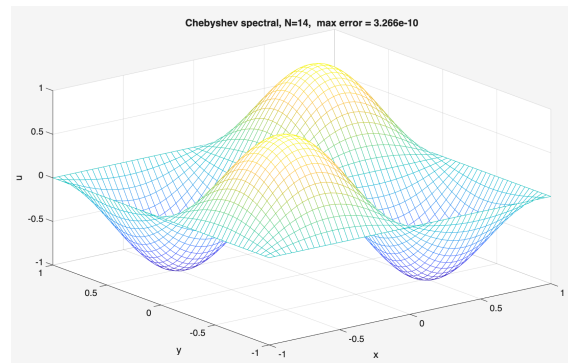


Figure 16: Solution of the Poisson equation (19) using the Pseudospectral method

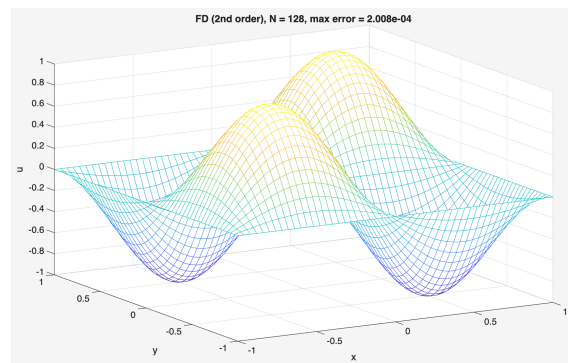


Figure 17: Solution of the Poisson equation (19) using the Finite difference

Poisson problem (19), as evidenced by the straight-line decay in the semilogarithmic plot of the max error versus the polynomial order N . In Figure 16 and Figure 17 we plot the pseudospectral and finite-difference numerical solutions, respectively.

Neumann and other Boundary Conditions

So far we have discussed only the simple homogeneous Dirichlet boundary condition, $u(\pm 1) = 0$, and periodic boundary conditions. However, many problems require more than these, and we will outline some of the techniques.

There are two basic approaches to boundary conditions for the Pseudospectral method [6, p. 135]:

- (I) Restrict attention to the interpolants that satisfy the boundary conditions;
- (II) Do not restrict the interpolants, but add additional equations to enforce the boundary conditions.

We have already seen the first method when dealing with homogeneous Dirichlet boundary conditions. To make more sense of the second method, let us consider (18) again with the mixed boundary conditions:

$$u_{xx} = e^{4x}, \quad -1 < x < 1, \quad u_x(-1) = u(1) = 0. \quad (22)$$

To enforce the boundary conditions, we actually need to impose a condition involving the first derivative. For the interior points, we can still use $\tilde{D}_N^{(2)}$ we introduced before, and for the boundary point x_N , it is natural to use the spectral differentiation matrix $D^{(1)}$. In other words, we are initially facing the following equation:

$$\sum_{j=0}^N (D_N^{(2)})_{kj} u(x_j) = f(x_k) \quad k \in \{1, \dots, N-1\}$$

Since we know $u(x_0) = 0$ from boundary conditions, we can delete the index $j = 0$ and get

$$\sum_{j=1}^N (D_N^{(2)})_{kj} u(x_j) = f(x_k) \quad k \in \{1, \dots, N-1\}.$$

To impose the Neumann condition $u_x(-1) = 0$, we write

$$\sum_{j=1}^N (D_N^{(1)})_{Nj} u(x_j) = 0.$$

We can apply the same idea in high dimensions, specifically an elliptic equation defined as follows:

$$-u_{xx} - u_{yy} + u = f \quad \text{on } \Omega = (-1, 1)^2 \quad (23)$$

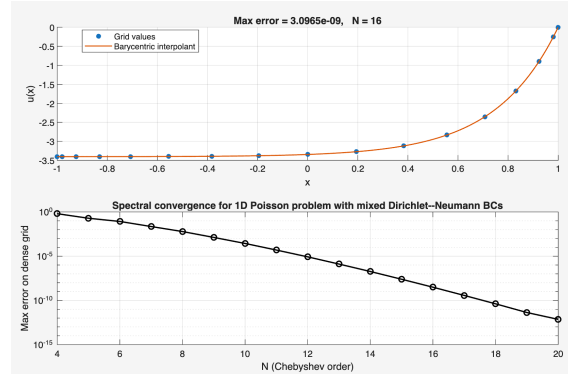


Figure 18: Solution of the mixed BC Equation (22)

subject to

$$\nabla u \cdot \nu = 0 \quad \text{on } \partial\Omega,$$

where f is a function such that $u = \cos(\pi x_1)(x_2^2 - 1)^2$ is the solution.

The absolute error of Equation (23) can be found in Figure 19 in which we see that an accuracy of order $O(10^{-13})$ can be achieved with only $N = 24$. The matrix construction in this case is more involved, because we cannot simply delete the first and last columns and rows as in the homogeneous Dirichlet boundary condition case, instead we have to modify the corresponding rows in order to impose the Neumann condition. However, in this case, the Galerkin method is easier to implement because the step of imposing boundary conditions is performed when using the variational formulation, which simplifies the integrals. The result is shown in Figure 20, similar to the Finite Difference case, the Finite Element computation attains only $O(10^{-3})$ accuracy, whereas the spectral method reaches $O(10^{-13})$ for $N = 24$.

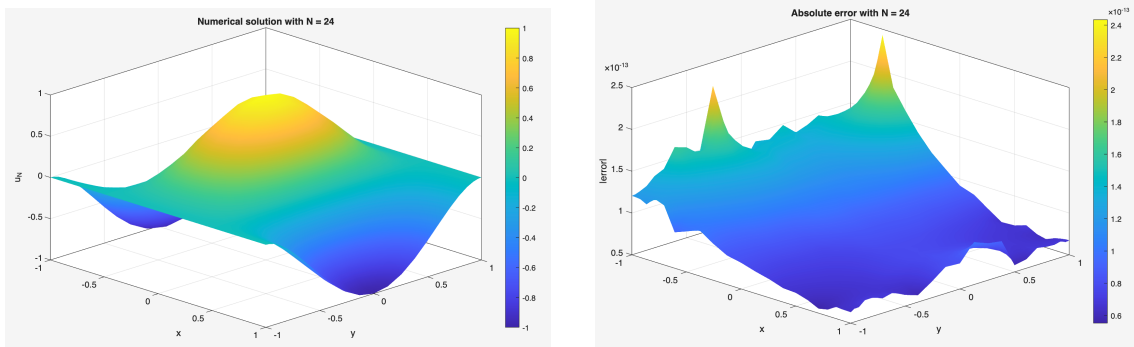


Figure 19: Numerical solution and absolute error for the spectral collocation method with $N = 24$. Left: numerical solution. Right: absolute error.

$$\begin{aligned}
\int_{\Omega} f\phi &= \int_{\Omega} (-\nabla \cdot (\nabla u) + u)\phi \quad \forall \phi \in H^1(\Omega) \\
&= \int_{\Omega} \nabla u \cdot \nabla \phi + \int_{\Omega} u\phi - \int_{\partial\Omega} (\nabla u \cdot \nu)\phi \\
&= \int_{\Omega} \nabla u \cdot \nabla \phi + \int_{\Omega} u\phi
\end{aligned}$$

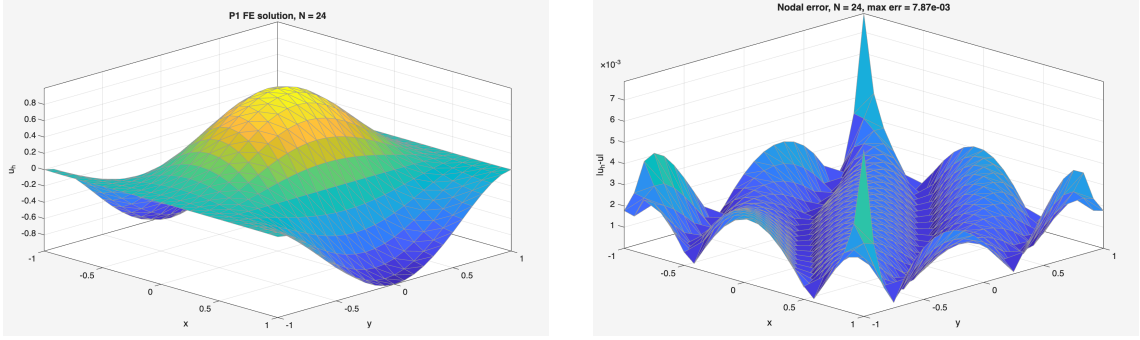


Figure 20: Numerical solution and nodal error for the Piecewise-linear finite element method with $N = 24$. Left: numerical solution. Right: nodal error.

7 Polar Coordinates

So far, we have only encountered square domains; it is natural to ask if the Pseudospectral method still works well with polar coordinates. A natural question is whether spectral accuracy can still be achieved in this setting?

In the plane, we have $x = r \cos(\theta)$ and $y = r \sin(\theta)$, and the Laplacian, $\Delta u = u_{xx} + u_{yy}$, turns into

$$u_{rr} + \frac{u_r}{r} + \frac{u_{\theta\theta}}{r^2}. \quad (24)$$

A standard problem defined on a unit disk would be

$$\begin{aligned}
\Delta u &= f(r, \theta), & \text{on } \Omega, \\
u &= 0, & \text{on } \partial\Omega.
\end{aligned} \quad (25)$$

The most common way to discretise the disk spectrally is to take a periodic Fourier grid on θ and a non-periodic Chebyshev grid in r . Specifically, the grid in the r -direction is transformed from the usual Chebyshev grid for $x \in [-1, 1]$ by $r = \frac{x+1}{2}$. The grids created in this way are convenient and widely used, but they do have some shortcomings. One is that while it is advantageous to have points clustered near the boundary, it may be wasteful and is certainly inelegant to devote extra grid points to the very small region near the origin, if the solution is smooth there. Another is that for time-dependent problems, these

small cells near the origin may force one to use excessively small time steps for numerical stability [6, p. 115].

Thus we will discuss a new method introduced by [4] that addresses these issues. The idea is to extend the radial coordinate to $r \in [-1, 1]$, and build a discrete system on the doubled grid. Finally, the numerical solution is obtained by restricting to the part with $r > 0$. However, this is not ‘well defined’, as each point (x, y) corresponds to two distinct points (r, θ) . To fix it, we need to impose an extra symmetry condition

$$u(r, \theta) = u(-r, (\theta + \pi) \bmod 2\pi). \quad (26)$$

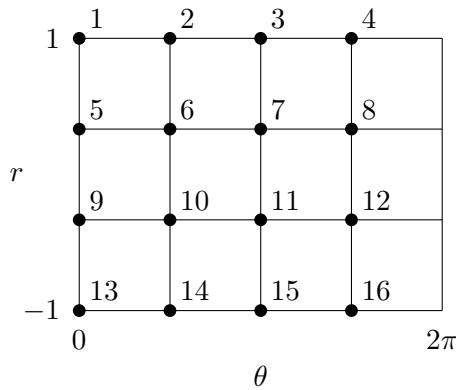
One might notice that at the origin it corresponds to infinitely many angular representations, but we can take N_r to be odd in r -direction to avoid it.

So, how do we compute the associated differential matrix?

Let N_r be the Chebyshev order and N_θ be the Fourier order. As we mentioned before, we take N_r to be odd, because choosing N_r odd ensures that $r = 0$ is not a grid point, so the singular coefficients $1/r$ and $1/r^2$ are never evaluated directly at the origin. And for simplicity we choose N_θ to be even, then we can use the formula we were given in Chapter 4. The associated second order differential matrices are D_{rr} and $D_{\theta\theta}$, respectively. Let D_r denote the first order differential matrix on the r -direction.

Our next goal is to combine these matrices together, based on the previous discussion on the 2D Laplacian operator on standard Euclidean coordinates, to represent Equation (24).

Consider a simple case with $N_r = 3$ and $N_\theta = 4$, and we number these points in the following order:



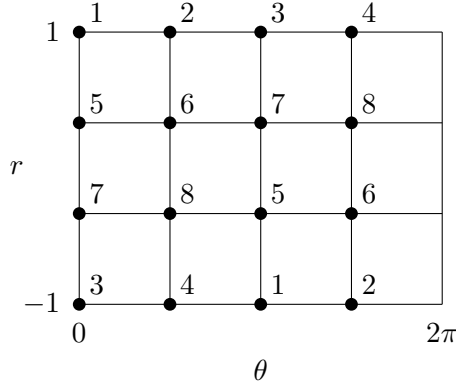
Note that the last column is left unnumbered because of periodicity. Since we reflect our domain to avoid crowding around the origin, only first eight points are what we need, and based on the previous discussion we have $(1, 5, 9, 13)$; $(2, 6, 10, 14)$; $(3, 7, 11, 15)$ and $(4, 8, 12, 16)$. Along each fixed angular direction, the radial second derivative is represented

by the same one-dimensional Chebyshev differentiation block. More precisely:

$$\begin{pmatrix} 1'' \\ 5'' \\ 9'' \\ 13'' \end{pmatrix} = \begin{pmatrix} D_{11} & D_{12} & D_{13} & D_{14} \\ D_{21} & D_{22} & D_{23} & D_{24} \\ D_{31} & D_{32} & D_{33} & D_{34} \\ D_{41} & D_{42} & D_{43} & D_{44} \end{pmatrix} \begin{pmatrix} 1 \\ 5 \\ 9 \\ 13 \end{pmatrix}$$

where 1 represents the function value at the point with index 1 and $1''$ represents the second derivative with respect to r at this point, and $D_{ij} = (D_{N_r}^{(2)})_{ij}$.

In particular, we have $1'' = D_{11} \cdot 1 + D_{12} \cdot 5 + D_{13} \cdot 9 + D_{14} \cdot 13$ but we only want $1''$ to be a linear combination of the first eight points, which means that we have to replace 9 and 13 by two of those points. It follows from the symmetry condition Equation (26) that $9 = 7$ and $13 = 3$. Then we can renumber these 16 points using only the first eight points and obtain:



After simplification, the second differential matrix with respect to r is

$$D_1 \oplus \begin{pmatrix} I & 0 \\ 0 & I \end{pmatrix} + D_2 \otimes \begin{pmatrix} 0 & I \\ I & 0 \end{pmatrix}$$

with $D_1 = \begin{pmatrix} D_{11} & D_{12} \\ D_{21} & D_{22} \end{pmatrix}$ and $D_2 = \begin{pmatrix} D_{14} & D_{13} \\ D_{24} & D_{23} \end{pmatrix}$. Notice the reverse order in D_2 .

A similar argument can be applied for the other two terms in Equation (24) and we get

$$(D_1 + RE_1) \otimes \begin{pmatrix} I & 0 \\ 0 & I \end{pmatrix} + (D_2 + RE_2) \otimes \begin{pmatrix} 0 & I \\ I & 0 \end{pmatrix} + R^2 \otimes D_{\theta\theta}$$

where I is the $N_{\theta/2} \times N_{\theta/2}$ identity and R is the diagonal matrix defined as

$$R = \text{diag}(r_j^{-1}), \quad 1 \leq j \leq \frac{N_r - 1}{2}$$

with r_j being Chebyshev points on r -direction, and E_i being defined the same as D_i by

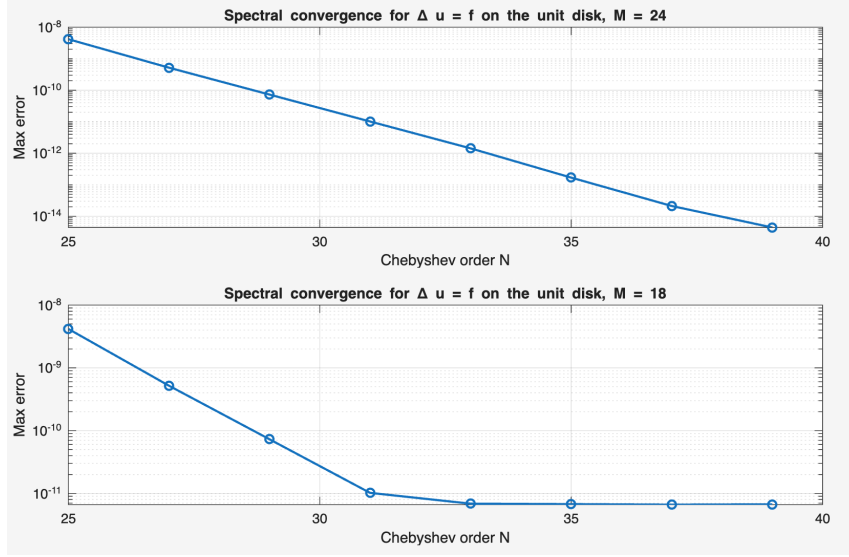


Figure 21: Spectral accuracy on polar coordinates

replacing $D_{N_r}^{(2)}$ by $D_{N_r}^{(1)}$ for each $i \in \{1, 2\}$.

To illustrate that the method retains spectral accuracy in polar coordinates, we consider Equation (25) with the following right-hand side:

$$f(r, \theta) = e^{-r^5 \cos \theta} \left(25r^8 \cos^2 \theta - 24r^3 \cos \theta + r^8 \sin^2 \theta \right) + e^{-r^3 \cos \theta} \left(-9r^4 \cos^2 \theta + 8r \cos \theta - r^4 \sin^2 \theta \right).$$

For this example, the exact solution is

$$u(r, \theta) = e^{-r^5 \cos \theta} - e^{-r^3 \cos \theta},$$

which satisfies the homogeneous Dirichlet boundary condition at $r = 1$. Figure 21 illustrates the convergence behaviour of the Fourier-Chebyshev discretisation for the Poisson equation on the unit disk. For fixed Fourier order M , the maximum error decays exponentially as the Chebyshev order N increases, indicating spectral accuracy in the radial direction. When M is sufficiently large (e.g. $M = 24$), this exponential decay persists down to near machine precision (10^{-14}). For smaller Fourier order (e.g. $M = 18$), the error initially decreases spectrally with N but eventually stagnates, showing that the overall accuracy is limited by the angular resolution.

8 Conclusion

Our exploration of the spectral method comes to an end. Let us recall what we have done so far. We have divided the discussion into two cases, depending on whether they are periodic or not. In both cases, pseudospectral differentiation is obtained by differentiat-

ing an interpolant in an appropriate global approximation space: polynomials on $[-1, 1]$ (Chebyshev) or trigonometric polynomials on $[0, 2\pi)$ (Fourier). Spectral accuracy follows from exponential coefficient decay for analytic functions, in other words, the more regular the function is, the more accurate the spectral method will be.

In contrast to the Finite Element method that adapts well with the irregular domains, the spectral method places strong restrictions on the shape of the domain. Classical tensor-product spectral methods are most natural on simple geometries: more complicated domains often require domain decomposition or other geometric transformations.

As discussed earlier, spectral methods share with the finite element method the general philosophy of approximating the solution in a finite-dimensional space, but they remain a distinct class of discretisation methods. Heuristically, spectral collocation is expressed as an ‘infinite-order’ finite difference [4]. Instead of using nearby points, we actually use all grid points, so the resulting differentiation matrices are dense rather than sparse, but far fewer grid points are typically needed to achieve high accuracy than in finite-difference methods.

Although we only obtain approximation of our function at grid points as always with spectral methods in the collocation setting, we have more information about the numerical solution than just point values. Implicitly, for the Chebyshev spectral methods, we are dealing with a polynomial interpolant $p(x)$ and this interpolant provides a natural continuous representation of the numerical solution. However, polynomial interpolation gets increasingly ill-conditioned at high degrees if we use *Vandermonde* matrix for interpolation, which implies the standard order `polyfit` in MATLAB is not adequate, instead we will use so-called *Barycentric Interpolation* [1].

Finally, a further issue, not pursued in this essay, is that Chebyshev collocation matrices are generally non-normal, so eigenvalues alone may not fully describe transient behaviour and stability. More details can be found in [7].

A Fast Fourier Transform revisited

In the following content, we will follow the same procedure as Tang does in [5] to introduce FFT, lemmas will be stated without proof, and detailed proofs can be found in [5, chapter. 1] or [2, chapter. 10].

Since we have already shown that in our setting, there is no difference for which N successive integers we take as wavenumber, so for simplicity, we will use

$$u_N(x) = \sum_{j=0}^{N-1} c_j e^{ijkx}$$

as our expression and $c_k = \frac{1}{N} \sum_{j=0}^{N-1} f(x_j) e^{-ikx_j}$, $\forall 0 \leq k \leq N-1$. Note that c_k defined here is $\frac{1}{2\pi}$ of $\hat{u}_k$ we defined in (13), which are just different conventions.

Lemma 1. *Let p and q be exponential polynomials of degree $N-1$ such that, for the points $y_j = \frac{\pi j}{N}$, we have*

$$p(y_{2j}) = f(y_{2j}), \quad q(y_{2j+1}) = f(y_{2j+1}), \quad 0 \leq j \leq N-1$$

Then the exponential polynomial of degree $\leq 2N-1$ that interpolates f at the points $y_j, 0 \leq j \leq 2N-1$, is given by

$$P(x) = \frac{1}{2}(1 + e^{iNx})p(x) + \frac{1}{2}(1 - e^{iNx})q(x - \frac{\pi}{N}). \quad (27)$$

This lemma actually tells us that we can first divide the whole grid into even grid and odd grid, interpolate them respectively and finally combine them together to get the original interpolant.

Lemma 2. *Let the coefficients of the polynomials described in Lemma (1) be as follows:*

$$p = \sum_{j=0}^{N-1} \alpha_j e^{ijx}, \quad q = \sum_{j=0}^{N-1} \beta_j e^{ijx}, \quad P = \sum_{j=0}^{N-1} \gamma_j e^{ijx}.$$

Then, for $0 \leq j \leq N-1$,

$$\gamma_j = \frac{1}{2}\alpha_j + \frac{1}{2}e^{-\frac{ij\pi}{N}}\beta_j, \quad \gamma_{j+N} = \frac{1}{2}\alpha_j - \frac{1}{2}e^{-\frac{ij\pi}{N}}\beta_j.$$

Continue from Lemma (1), this lemma (2) actually tells us how to compute the coefficients of the interpolant.

Let $R(N)$ denote the minimum number of multiplications necessary to compute the coeffi-

cients in an interpolating exponential polynomial for the set of points $\{\frac{2j\pi}{N} : 0 \leq j \leq N-1\}$. Then we have

$$R(2N) \leq 2R(N) + 2N. \quad (28)$$

It is seen that $R(2N)$ is the minimum number of multiplications necessary to compute γ_j , and $R(N)$ is the minimum number of multiplications necessary to compute α_j or β_j . By Lemma (2), the coefficients γ_j can be obtained from α_j and β_j at the cost of $2N$ multiplications. Indeed, we require N multiplications to compute $\frac{1}{2}\alpha_j$ for $0 \leq j \leq N-1$, and another N multiplications to compute $(\frac{1}{2}e^{-\frac{ij\pi}{N}})\beta_j$ for $0 \leq j \leq N-1$.

It follows from (28) that $R(2^m) \leq m2^m$. If $N = 2^m$ then the cost of computing the interpolating exponential polynomials obey the inequality

$$R(N) \leq N \log_2(N).$$

However, this argument does not merely state that the FFT has $O(N \log(N))$ complexity; rather, it establishes an explicit recurrence for the minimal number of multiplications required to compute the interpolating exponential polynomial. The subsequent tree diagram illustrates how this recurrence is realised algorithmically.

In fact, the algorithm that repeatedly carries out the procedure in Lemma (1) is the fast Fourier transform.

Tree diagram

The content of Lemma (1) can be interpreted in terms of two linear operators, L_N and T_h . For any f , let $L_N f$ denote the exponential polynomial of degree $N-1$ that interpolates f at the nodes $\frac{2j\pi}{N}$ for $0 \leq j \leq N-1$. Let T_h be a *translation operator* defined by $T_h(f)(x) = f(x+h)$. Then $P = L_{2N}f$, $p = L_N f$ and $q = L_N T_{\frac{\pi}{N}} f$. The conclusion of the previous two Lemmas is that $L_{2N}f$ can be obtained efficiently from $L_N f$ and $L_N T_{\frac{\pi}{N}} f$.

We now aim to establish one version of the fast Fourier transform algorithm for computing $L_N f$ with $N = 2^m$. We define

$$P_k^{(n)} = L_{2^n} T_{\frac{2k\pi}{N}} f, \quad 0 \leq n \leq m, 0 \leq k \leq 2^{m-n} - 1. \quad (29)$$

An alternative description of $P_k^{(n)}$ is as the exponential polynomial of degree $2^n - 1$ that interpolates f in the following way:

$$P_k^{(n)}\left(\frac{2\pi j}{2^n}\right) = f\left(\frac{2\pi k}{N} + \frac{2\pi j}{2^n}\right), \quad 0 \leq j \leq 2^n - 1.$$

We can illustrate in a tree diagram (in Figure 22) how the exponential polynomials $P_k^{(n)}$

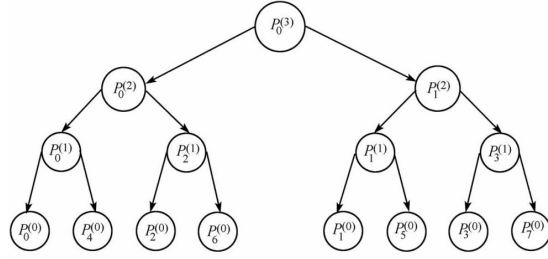


Figure 22: An illustration of a tree diagram [5, p. 32]

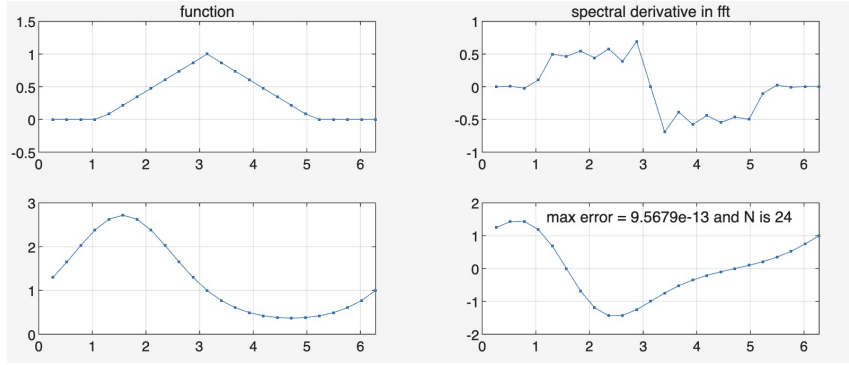


Figure 23: Repetition of Figure (2) based on FFT

are related. Suppose that our objective is to compute

$$P_0^{(3)} = L_8 f = \sum_{k=0}^7 \hat{u}_k e^{ikx}.$$

The fast Fourier transform is calculated by MATLAB's programs `fft` and `ifft`. In our notation, MATLAB stores wavenumbers in the order $0, 1, \dots, \frac{N}{2}, -\frac{N}{2} + 1, \dots, -1$.

Let us consider the hat function $1 - |\frac{x-\pi}{2}|$ and the smooth function $e^{\sin(x)}$ again using fast Fourier transform instead of differential matrices. We can see in Figure 23 that the results turn out to be quite similar.

References

- [1] Jean-Paul Berrut and Lloyd N Trefethen. Barycentric lagrange interpolation. *SIAM review*, 46(3):501–517, 2004.
- [2] John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.
- [3] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [4] Bengt Fornberg. *A practical guide to pseudospectral methods*, volume 1. Cambridge university press, 1998.
- [5] Tao Tang. *Spectral and high-order methods with applications*. Science Press Beijing, 2006.
- [6] Lloyd N Trefethen. *Spectral methods in MATLAB*. SIAM, 2000.
- [7] Lloyd Nicholas Trefethen. Finite difference and spectral methods for ordinary and partial differential equations. 1996.
- [8] J. A. C. Weideman and S. C. Reddy. A MATLAB differentiation matrix suite. *ACM Transactions on Mathematical Software*, 26(4):465–519, 2000.